

ARQUITECTURA MIPS

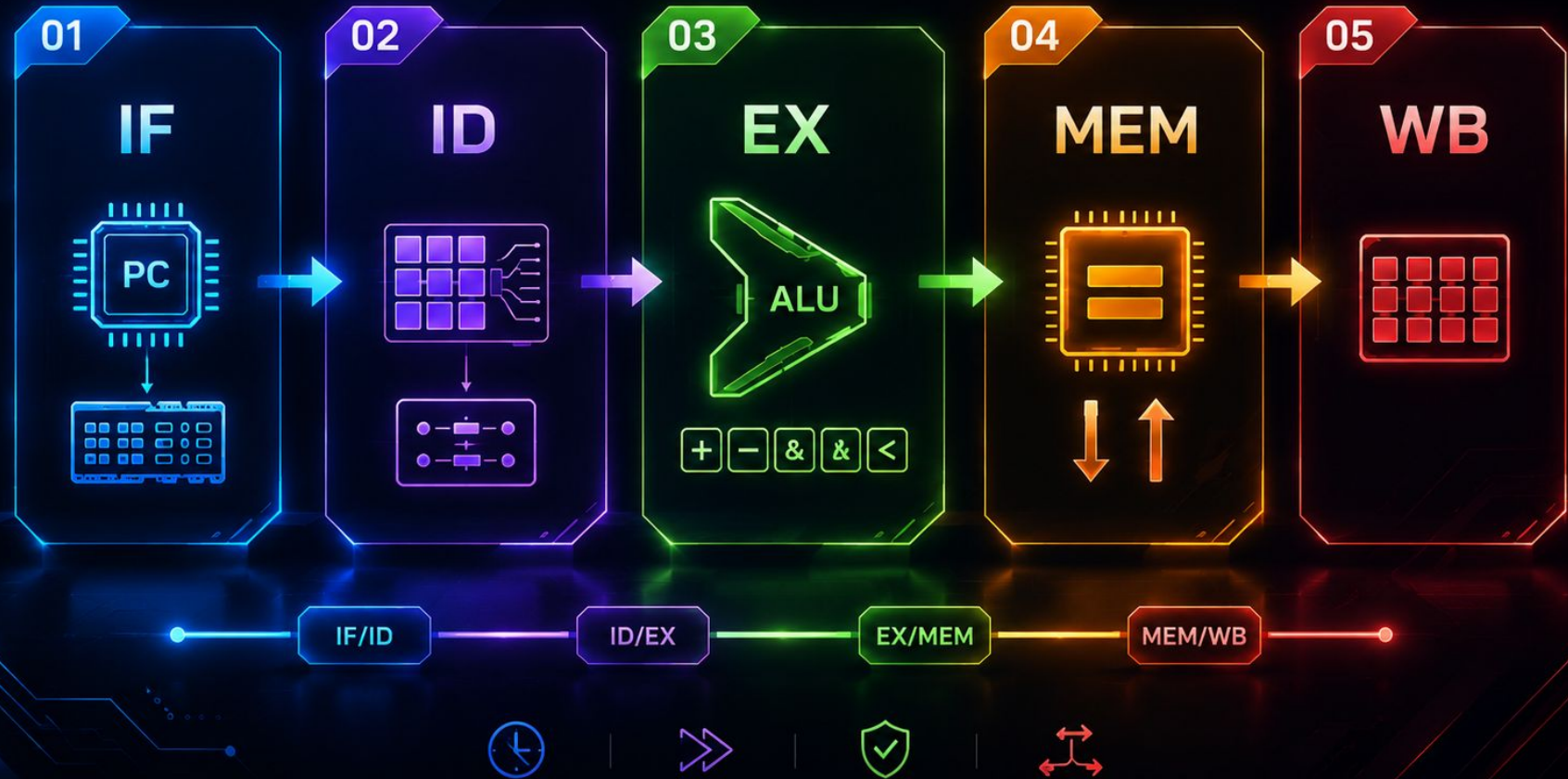
IMPLEMENTACIÓN DE ALGORITMOS
DE ORDENAMIENTO Y BÚSQUEDA

INTEGRANTES:

- HERNÁNDEZ ZÚÑIGA ROBERTO
- LÓPEZ LÓPEZ KARLA VALERIA
- ORTIZ SEGURA EMMANUEL ALEJANDRO
- SEGURA GUTIÉRREZ AARÓN RICARDO



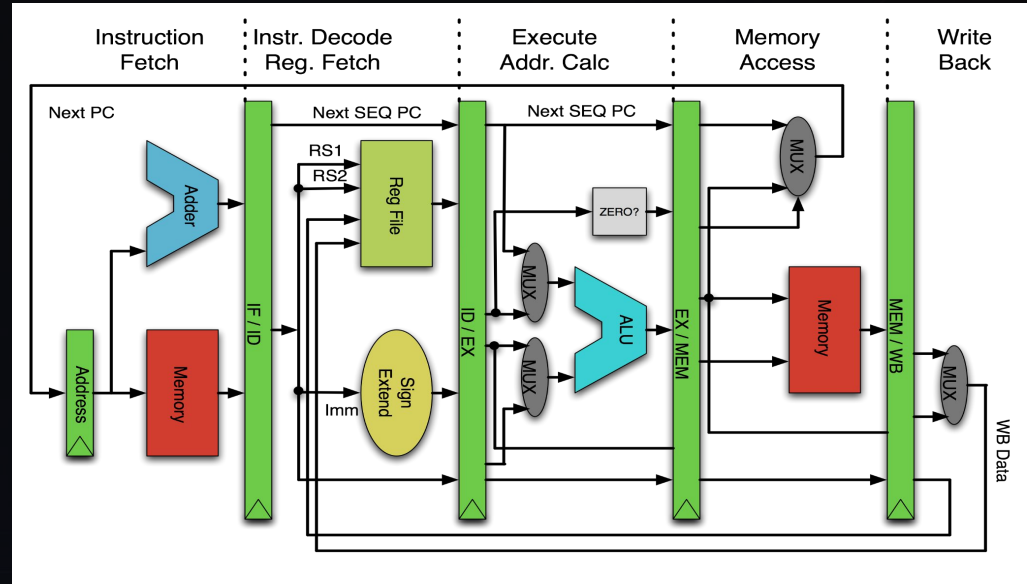
Visión General de la Arquitectura



PIPELINE DE 5 ETAPAS

Aquí observamos el clásico pipeline MIPS, a lo largo de esta presentación veremos cómo esto se traslada directamente a la realidad de nuestro proyecto.

Mas adelante, mostraremos qué módulos implementamos en código para construir cada uno de estos bloques



Instruction Fetch

PC

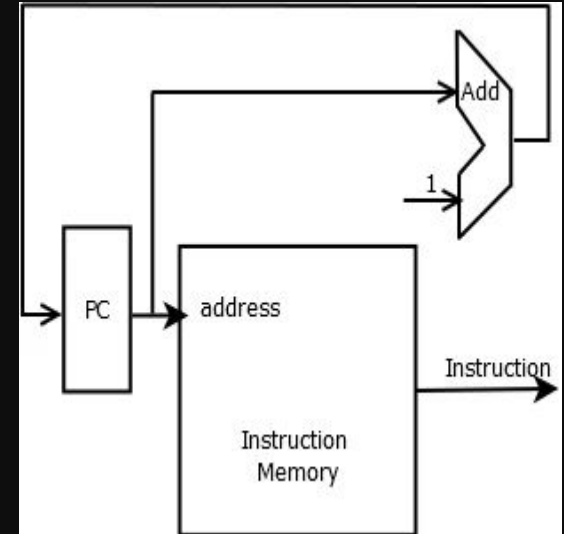
Guarda la dirección actual
de ejecución

MEM_I

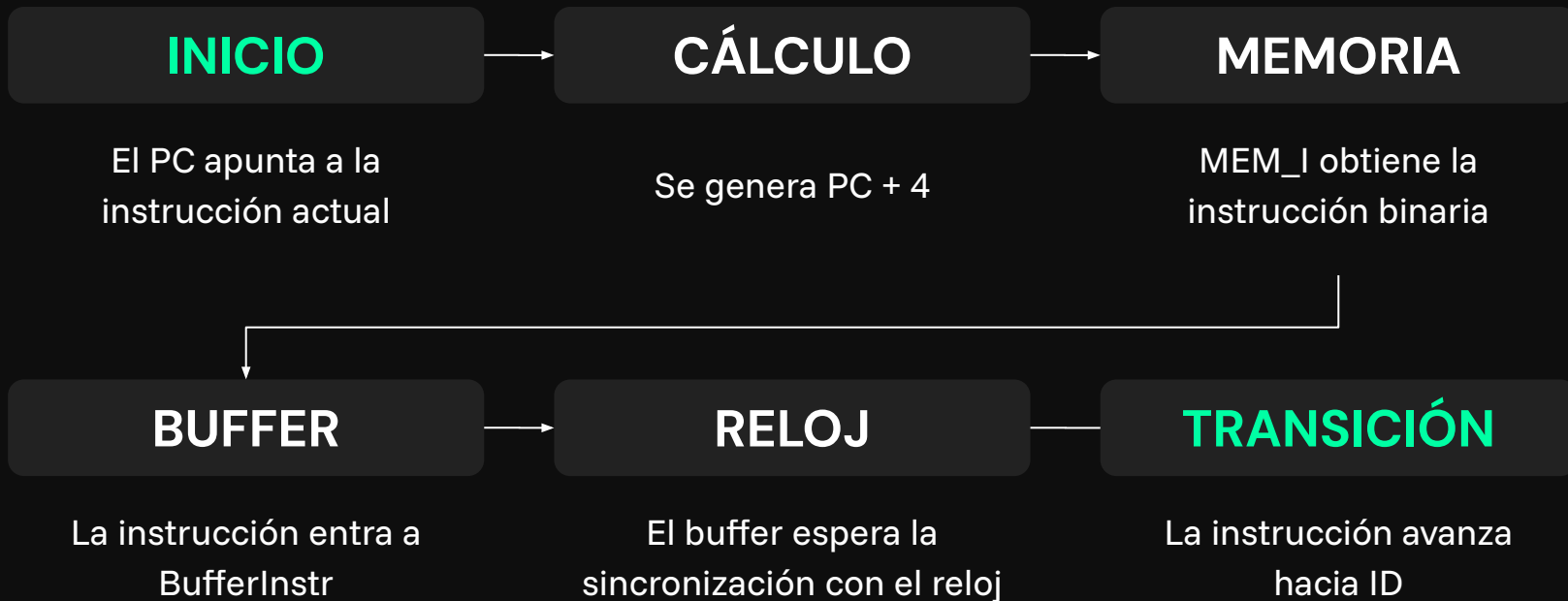
Obtiene la instrucción
desde memoria

BUFFER INSTR

Guarda instrucción y PC + 4
para la siguiente etapa



SECUENCIA IF



Instruction Decode

U_Control

Genera señales de control.

Sign Extend

Extiende immediatos a 32 bits.

BancoReg

Lee registros fuente rs y rt

Buffer Datos

Guarda datos y control para EX.

R (Register) Format:

Opcode (6)	Rs (5)	Rt (5)	Rd (5)	Shamt (5)	Funct (6)
---------------	-----------	-----------	-----------	--------------	--------------

Most arithmetic and logic instructions (except 'immediate')

I (Immediate) Format:

Opcode (6)	Rs (5)	Rt (5)	16-bit Immediate value (16)
---------------	-----------	-----------	---------------------------------------

Data Transfer, Immediate, and Cond. Branch instructions

J (Jump) Format:

Opcode (6)	26-bit word address (26)
---------------	------------------------------------

Unconditional Jump instructions

SECUENCIA ID



EXECUTE

ALU_Control

Selecciona operación de
ALU

ALU

Ejecuta las operaciones

MUX2_1_32

Selecciona operando de la
ALU

MUX2_1_5

Selecciona registro destino

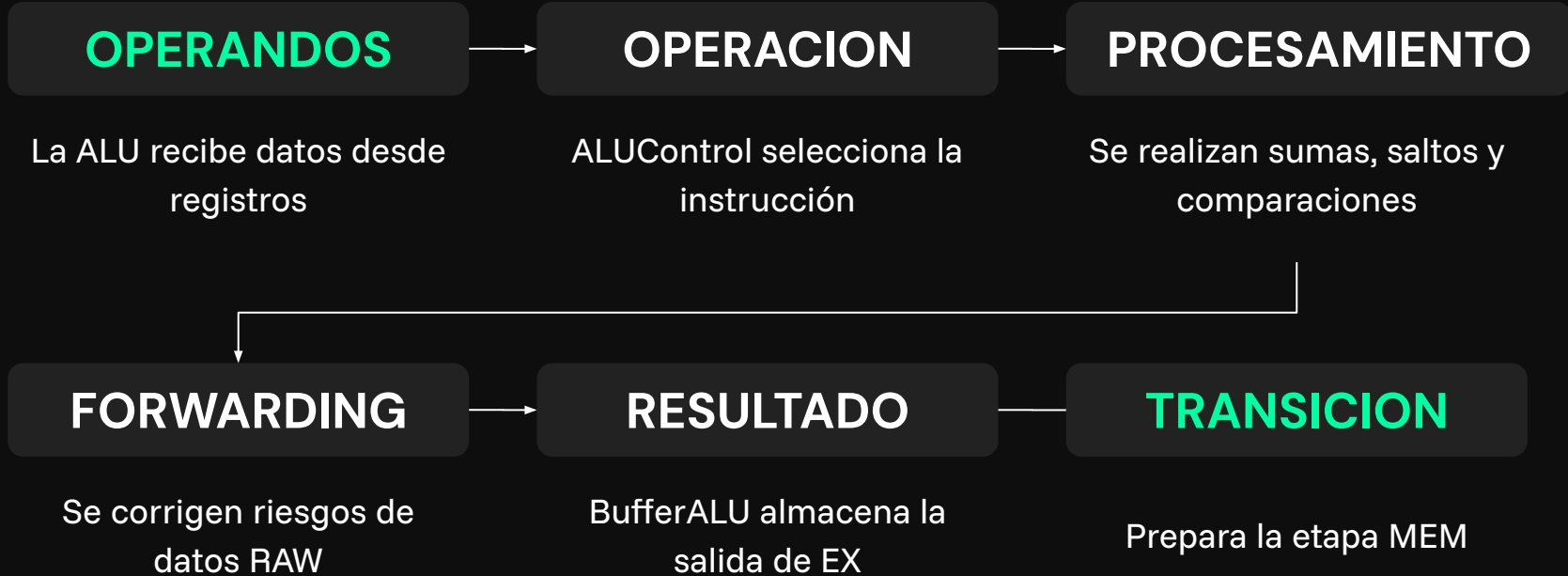
Forwarding_Unit

Resuelve hazards de datos

BufferALU

Guarda resultados para
MEM

SECUENCIA EXECUTE



MEM

MEM_D

Lee o escribe datos en
memoria

BufferFinal

Guarda resultados para WB

SECUENCIA MEM

DIRECCION

Se recibe la dirección
calculada

LECTURA

LW obtiene información
desde Mem_D

ESCRITURA

SW guarda datos en
memoria

CONTROL

Las señales determinan
lectura o escritura

BUFFER

BufferFinal prepara etapa
WB



WRITE BACK

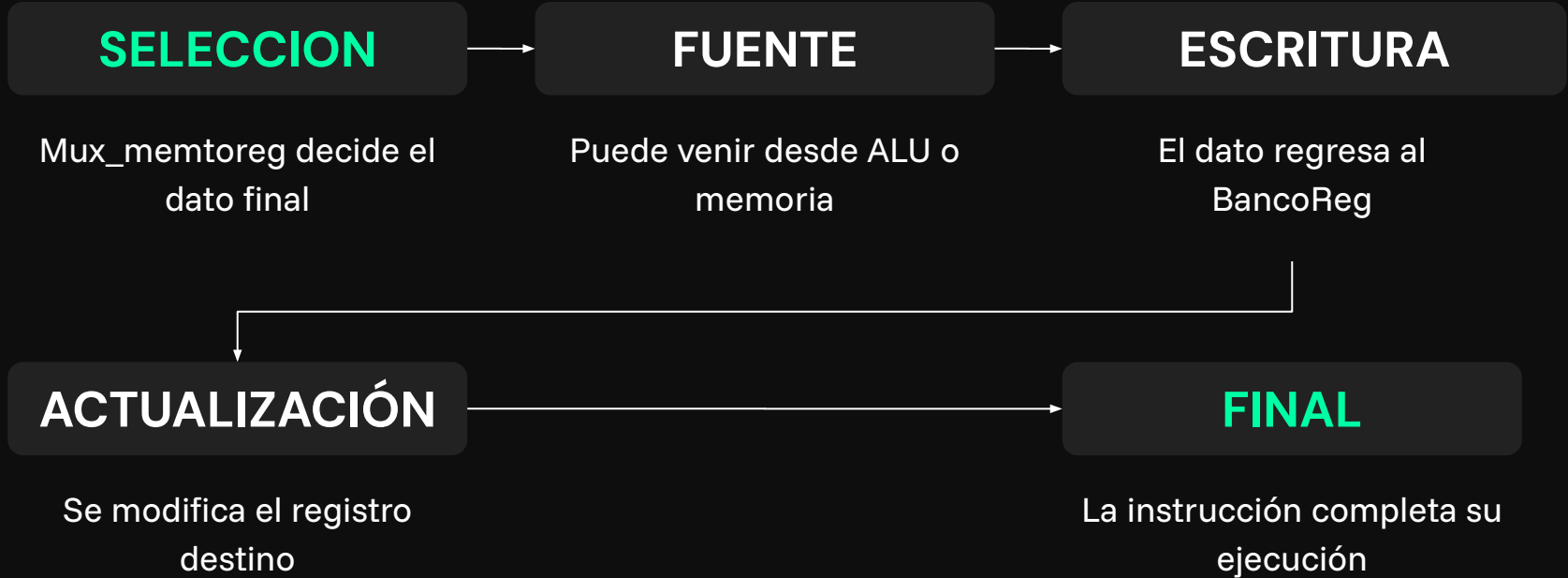
Mux2_1_32

Selecciona dato final

BancoReg

Escribe resultado en
registros

SECUENCIA WB



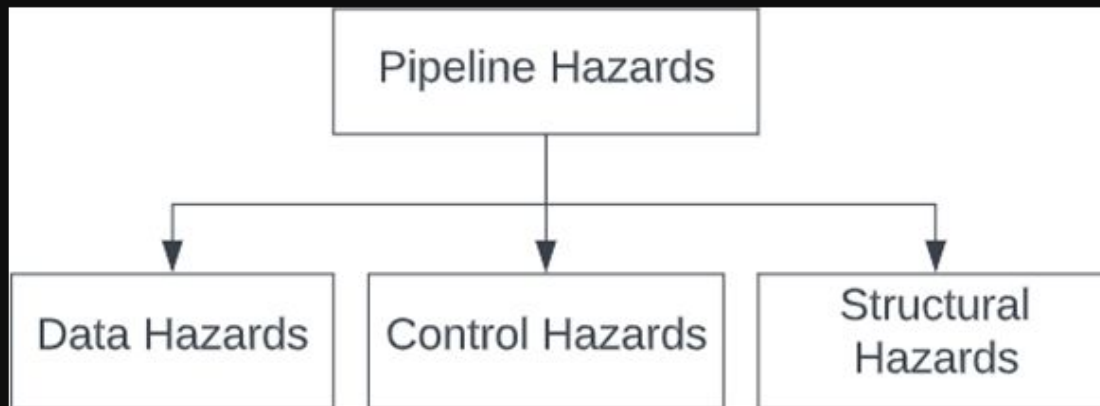
El súper algoritmo

Para poner a prueba nuestro diseño, escogimos el algoritmo de búsqueda sencillo por excelencia, búsqueda binaria. Consiste en ir a la mitad de el conjunto de datos y comparar con el elemento buscado, si es mayor descarta los de la izquierda y se queda con los de la derecha o se queda con los de la izquierda en caso contrario, se repite hasta que lo encuentra.

El problema es que para ello requiere que los datos estén ordenados, pero eso lo solucionamos con otro algoritmo, el algoritmo de ordenamiento de burbuja:) es un método sencillo que revisa una lista de elementos comparándolos de dos en dos. Si el elemento de la izquierda es mayor que el de la derecha, los intercambia. Combinando ambos tenemos el “súper algoritmo”

Ensamblador del **algoritmo**

En las instrucciones se ve frecuente el uso de la instrucción Nop (No operation) estos sirven para evitar Riesgos del Pipeline (Hazards). Al insertar nop, le damos tiempo al procesador de terminar de guardar un dato en la memoria o en un registro antes de que la siguiente instrucción intente usarlo.



Para profundizar más: GeeksforGeeks. (2025, November 11). Pipeline hazards. GeeksforGeeks.
<https://www.geeksforgeeks.org/computer-organization-architecture/pipeline-hazards/>

Fase de configuración

Instrucción en Ensamblador	¿Qué hace aritméticamente?	Explicación del Algoritmo
addi \$s0, \$zero, 0	Suma $0 + 0$ y lo guarda en \$s0.	Fija el registro base del arreglo en la dirección de memoria 0.
addi \$s1, \$zero, 10	Suma $0 + 10$ y lo guarda en \$s1.	Define el tamaño del arreglo ($n = 10$).
addi \$s2, \$zero, 45	Suma $0 + 45$ y lo guarda en \$s2.	Define el valor objetivo a buscar al final ($X = 45$).

Fase de Ordenamiento (Bubble Sort)

Etiqueta / Instrucción	Operación Interna	Explicación del Algoritmo
addi \$t0, \$zero, 0	\$t0 = 0	Inicializa el contador del ciclo externo (i = 0).
CICLO_EXTERNO:		-- Inicio de la pasada por el arreglo --
addi \$t8, \$s1, -1	\$t8 = 10 - 1 = 9	Calcula el límite superior (n - 1).
slt \$t9, \$t0, \$t8	Si \$t0 < \$t8, entonces \$t9 = 1, sino 0.	Comprueba si i < (n - 1).
beq \$t9, \$zero, INICIO_BUSQUEDA	Si \$t9 == 0, salta a INICIO_BUSQUEDA.	Si i llegó al límite, el arreglo ya está ordenado, se sale del ciclo
addi \$t1, \$zero, 0	\$t1 = 0	Inicializa el contador del ciclo interno (j = 0).
CICLO_INTERNO:		-- Recorre los elementos no ordenados --
sub \$t7, \$t8, \$t0	\$t7 = \$t8 - \$t0	Límite interno = (n - 1) - i. Optimiza para no revisar los números que ya subieron al final.
slt \$t9, \$t1, \$t7	Si \$t1 < \$t7, \$t9 = 1, sino 0.	Comprueba si j < limite_interno.

Fase de Ordenamiento (Bubble Sort)

beq \$t9, \$zero, FIN_INTERNO	Si \$t9 == 0, salta a FIN_INTERNO.	Si j llegó al límite, termina esta pasada y avanza la i.
(Cálculo de Memoria)		-- Multiplicación por 4 (Alineamiento) --
add \$t4, \$t1, \$t1	\$t4 = j + j (equivale a $j * 2$)	MIPS avanza de 4 en 4 bytes. Para acceder al índice j, multiplicamos por 4 usando sumas.
add \$t4, \$t4, \$t4	\$t4 = 2j + 2j (equivale a $j * 4$)	Tenemos el "offset" o desplazamiento exacto en bytes.
add \$t5, \$s0, \$t4	\$t5 = dir_base + offset	\$t5 ahora tiene la dirección física de memoria de A[j].
(Lectura y Comparación)		-- Extracción de datos RAM --
lw \$s3, 0(\$t5)	\$s3 = Memoria[\$t5]	Carga en el registro \$s3 el valor actual de A[j].
lw \$s4, 4(\$t5)	\$s4 = Memoria[\$t5 + 4]	Carga en el registro \$s4 el valor adyacente A[j+1].
slt \$t6, \$s4, \$s3	Si \$s4 < \$s3, \$t6 = 1, sino 0.	¿El elemento de la derecha es MENOR que el de la izquierda?
beq \$t6, \$zero, NO_SWAP	Si \$t6 == 0, salta a NO_SWAP.	Si el de la derecha es mayor, están en buen orden. No intercambia nada

Fase de Ordenamiento (Bubble Sort)

(Intercambio - Swap)		-- Volteo de datos en RAM --
sw \$s4, 0(\$t5)	Memoria[\$t5] = \$s4	Sobrescribe A[j] con el valor de A[j+1].
sw \$s3, 4(\$t5)	Memoria[\$t5 + 4] = \$s3	Sobrescribe A[j+1] con el valor de A[j].
NO_SWAP:	(<i>no hay cambio</i>)	
addi \$t1, \$t1, 1	\$t1 = \$t1 + 1	Incrementa j++.
j CICLO_INTERNO	Salta incondicionalmente.	Regresa a evaluar el siguiente par de números.
FIN_INTERNO:		
addi \$t0, \$t0, 1	\$t0 = \$t0 + 1	Incrementa i++.
j CICLO_EXTERNO	Salta incondicionalmente.	Inicia una nueva pasada por el arreglo.

Fase de **Búsqueda** Binaria

Etiqueta / Instrucción	Operación Interna	Explicación del Algoritmo
INICIO_BUSQUEDA:		-- Preparación --
addi \$v0, \$zero, -1	\$v0 = -1	Guarda -1 en \$v0. Si el número no se encuentra, este valor será el resultado final.
addi \$t0, \$zero, 0	\$t0 = 0	Límite Izquierdo de búsqueda = 0.
addi \$t1, \$s1, -1	\$t1 = 10 - 1 = 9	Límite Derecho de búsqueda = 9.
BUSQUEDA:		-- Bucle Principal (while izq <= der) --
slt \$t2, \$t1, \$t0	Si \$t1 < \$t0, \$t2 = 1, sino 0.	¿Se cruzaron los límites? (Derecha menor que Izquierda).
bne \$t2, \$zero, FIN	Si \$t2 != 0, salta a FIN.	Si se cruzaron, significa que revisamos todo y no está el número. Ahí acaba
(Cálculo del Medio)		
add \$t3, \$t0, \$t1	\$t3 = izq + der	Suma los dos límites.

Fase de **Búsqueda** Binaria

srl \$t4, \$t3, 1	\$t4 = \$t3 >> 1	Shift Right Logical. Desplazar 1 bit a la derecha equivale a dividir entre 2. Se calcula el indice medio
(Cálculo de Memoria)		
add \$t5, \$t4, \$t4 add \$t5, \$t5	\$t5 = medio * 4	Igual que en Bubble Sort, multiplicamos por 4 para alinear los bytes.
add \$t6, \$s0, \$t5	\$t6 = base + offset	Dirección física de A[medio].
lw \$s3, 0(\$t6)	\$s3 = Memoria[\$t6]	Extrae el valor de la RAM en la posición media.
(Comprobación)		
beq \$s3, \$s2, EXITO	Si A[medio] == Valor(\$s2), salta.	Si lo encuentra, va a la etiqueta EXITO.
slt \$t7, \$s3, \$s2	Si A[medio] < Valor, \$t7 = 1.	Si no es igual, revisa: ¿El número en el medio es menor que el buscado?
beq \$t7, \$zero, IZQUIERDA	Si \$t7 == 0, salta a IZQUIERDA.	Si el número en el medio es mayor o igual, la respuesta está en la mitad izquierda.
(Mover Límite DERECHA)	(Si la CPU llega aquí es porque A[medio] < Valor)	

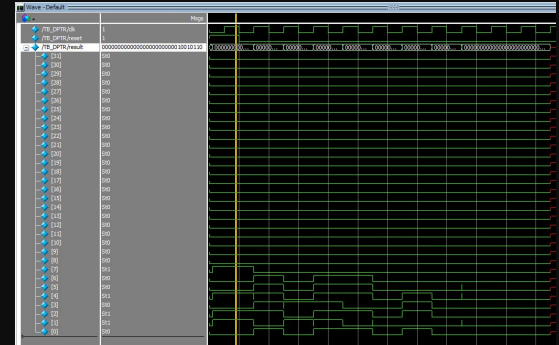
Fase de **Búsqueda** Binaria

addi \$t0, \$t4, 1	$\$t0 = \text{medio} + 1$	Mueve el límite Izquierdo. Ignoramos la mitad inferior.
j BUSQUEDA	Salta incondicionalmente.	Repite la búsqueda con los nuevos límites.
IZQUIERDA:		
addi \$t1, \$t4, -1	$\$t1 = \text{medio} - 1$	Mueve el límite Derecho. Ignoramos la mitad superior.
j BUSQUEDA	Salta incondicionalmente.	Repite la búsqueda con los nuevos límites.
EXITO:		
add \$v0, \$zero, \$t4	$\$v0 = 0 + \text{medio}$	Guarda el índice donde se encontró el número en \$v0. Sobrescribe el -1 inicial.
FIN:		
j FIN	PC = PC de FIN	Trampa final (bucle infinito). Evita que el procesador intente seguir ejecutando memoria vacía.

Resultados de la simulación

WAVE

Nuestro diseño es capaz de leer las instrucciones descritas en el ensamblador y convertidas a binario con el conversor de python. Luego con el módulo del test bench para el DPTR se le dio valor a los registros utilizados en las instrucciones tipo R del ensamblador para la fase 1.

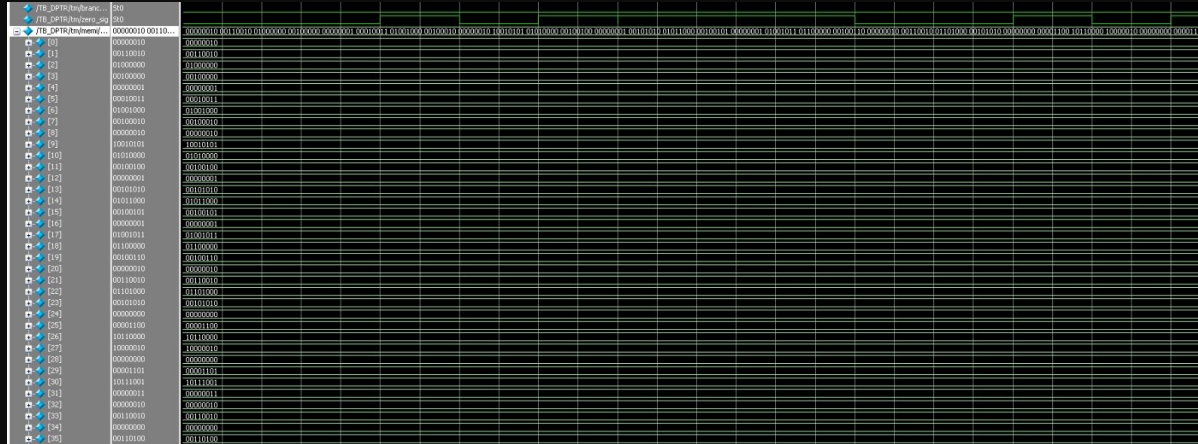


Memory List

Aquí puede observarse el memory list con los valores de cada registro después de correr la simulación, los valores corresponden a los resultados de cada operación.

Memory Data - /TB_DPTR/tm/dp/BR/regs - Default		
0	0	0
2	0	0
4	0	0
6	0	0
8	150	125
10	10	127
12	117	0
14	0	0
16	0	100
18	50	25
20	30	10
22	29	0
24	0	0
26	0	0
28	0	0
30	0	0

Resultados de la simulación

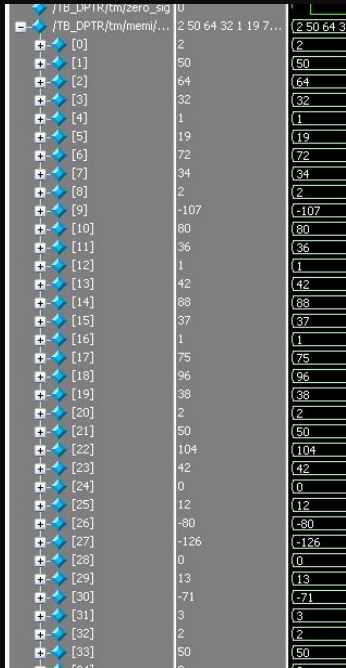


Wave

Para la fase 2, nuestro procesador fue capaz de entender las instrucciones tipo I, luego de hacer modificaciones en nuestro data path, en la simulación se muestra cómo se guardan en la memoria

Simulación en ModelSim donde se observan datos correctos en binario y decimal.

Resultados de la simulación



0	2	2
1	50	50
2	64	64
3	32	32
4	1	1
5	19	19
6	72	72
7	34	34
8	2	2
9	-107	-107
10	80	80
11	36	36
12	1	1
13	42	42
14	88	88
15	37	37
16	1	1
17	75	75
18	96	96
19	38	38
20	2	2
21	50	50
22	104	104
23	42	42
24	0	0
25	12	12
26	-80	-80
27	-126	-126
28	0	0
29	13	13
30	-71	-71
31	3	3
32	2	2
33	50	50

00000000	00000004	00000008	0000000c	00000010
00000004	00000008	0000000c	00000010	00000014
02324020	01134822	02955024	012a5825	014b6026
00000000	02324020	01134822	02955024	012a5825

En el siguiente apartado se muestra cómo los buffers agregados desde la fase 2 funcionan correctamente, copiando cada valor para el siguiente bloque en cada ciclo de reloj.

El procesador funciona correctamente para la fase 2.

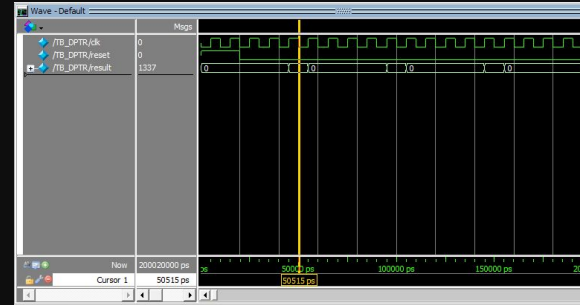
Resultados de la simulación

TB_DPTR

Cada 5 flancos de subida el resultado de la señal result es 1337, lo que corresponde al resultado de la instrucción addi, luego vuelve a 0, y sigue un ciclo infinito por la instrucción de salto, hasta terminar el tiempo de simulación.

Memory List

Podemos observar el memory list de nuestro Banco de Registros, con el resultado correcto para la instrucción addi (1337 en el registro 8).

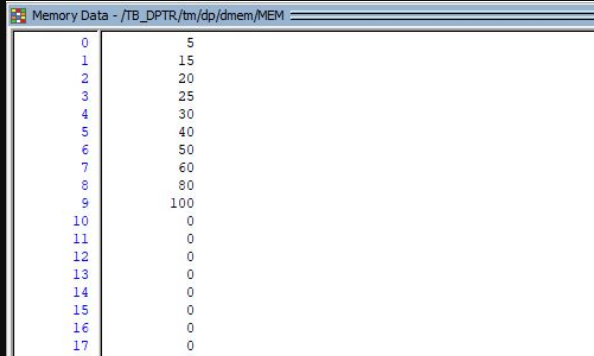


0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	1337
9	0
10	0
11	0
12	0
13	0
14	0

Resultados de la simulación

Memoria de Datos

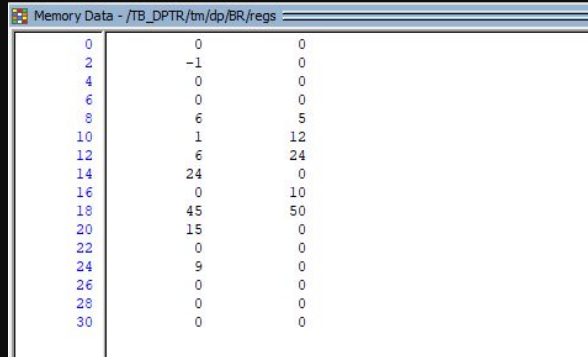
Memory list de la memoria de datos, se realizó de manera correcta el Bubble Sort, recibiendo como resultado el arreglo de manera ordenada, listo para la búsqueda binaria.



0	5
1	15
2	20
3	25
4	30
5	40
6	50
7	60
8	80
9	100
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	0

Banco de registros

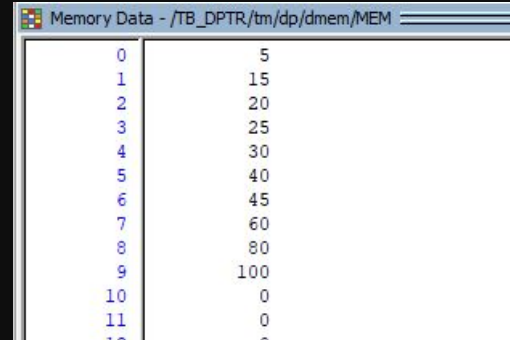
Memory list del Banco de Registros, lo más importante a notar es que en el registro 2 se encuentra el valor -1 (caso de fracaso).



0	0	0
2	-1	0
4	0	0
6	0	0
8	6	5
10	1	12
12	6	24
14	24	0
16	0	10
18	45	50
20	15	0
22	0	0
24	9	0
26	0	0
28	0	0
30	0	0

Banco de registros

Para el caso de éxito se inserta el valor 45 en el arreglo, y el Bubble Sort vuelve a funcionar.



0	5
1	15
2	20
3	25
4	30
5	40
6	45
7	60
8	80
9	100
10	0
11	0
12	0

Resultados de la simulación

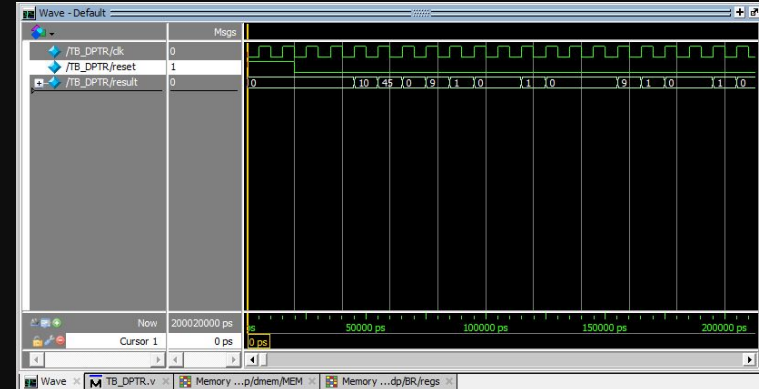
Banco de Registros

Caso de éxito muestra el resultado 6 en el registro 2, es decir, la posición en que se encuentra el número que buscábamos, dirección número 6 de la memoria de datos.

Memory Data - /TB_DPTR/tm/dp/BR/regs		
0	0	0
2	6	0
4	0	0
6	0	0
8	6	6
10	0	12
12	6	24
14	24	1
16	0	10
18	45	45
20	15	0
22	0	0
24	9	0
26	0	0
28	0	0
30	0	0

Wave

Ventana Wave muestra las señales de cada instrucción que se va mandando, 0 (inicialización), 10 (tamaño del arreglo), 45 (número a buscar), etc.



¿Preguntas?

Referencias:

Patterson & Hennessy, "Computer Organization and Design: The Hardware/Software Interface"

MIPS32 Architecture for Programmers Volume

Proyecto de **Arquitectura de Computadoras**